

detected by the sensor. The *SensorEvent* parameter will hold the values for all three axes.

```
@Override
public void onSensorChanged(SensorEvent event) {
    if (event.sensor.getType() != Sensor.TYPE_
ACCELEROMETER)
        return;
    // X-axis
    if (event.values[0] > 0) {
        mSensorValuesLeft.setText("X-axis (+)ve: "
            + Integer.toString((int) event.values[0]));
    } else if (event.values[0] < 0) {
        mSensorValuesRight.setText("X-axis (-)ve:: "
            + Integer.toString(((int) event.values[0])
* -1));
    }

    float y = event.values[1];
    if (y > 0) {
        mSensorValuesUp.setText("Y-axis (+)ve: "
            + Integer.toString((int) y));
    } else {
        mSensorValuesDown.setText("Y-axis (-)ve: "
            + Integer.toString((int) y * -1));
    }

    float z = event.values[2];
    if (z > 0) {
        mSensorValuesZUp.setText("Z-axis (+)ve: "
            + Integer.toString((int) z));
    } else {
        mSensorValuesZDown.setText("Z-axis (-)ve: "
            + Integer.toString((int) z * -1));
    }
}
```

The logic is very simple: the *SensorEvent* variable is an array holding 3 values corresponding to the X axis (element 0) and to the Z axis (element 2) respectively. Use simple *if-else* logic to set the values to their corresponding *TextViews*. If the value is positive, then assign it to the positive value of the axis, otherwise set the negative value of the axis. The negative value is multiplied by (-1) so that it will be displayed as a (+)ve value for the (-)ve axis.

```
@Override
protected void onPause() {
    // TODO Auto-generated method stub
    super.onPause();
    mSensorManager.unregisterListener(this);
}
```

```
@Override
protected void onResume() {
    // TODO Auto-generated method stub
    super.onResume();
    mSensorManager.registerListener(this, mAccelerometer,
        SensorManager.SENSOR_DELAY_UI);
}

@Override
protected void onStop() {
    // TODO Auto-generated method stub
    super.onStop();
    mSensorManager.unregisterListener(this);
}
```

As discussed earlier, do not listen to sensors all the time, because it could lead to a serious battery drain—so register in *onResume()* and un-register in *onPause()* and *onStop()*. Now, extend this app by showing a list of sensors such that when a user clicks one, the respective sensor demo app is opened. To accomplish the above task, I will create another activity, and show a list of sensors supported by the device. (Reference for the concepts below can be found in my previous article of *FragmentDemo*, published in LFY, May 2011.)

```
mSensorManager = (SensorManager) getSystemService(Context.
SENSOR_SERVICE);
mSensorList = mSensorManager.getSensorList(Sensor.TYPE_
ALL);
ArrayList<String> mSensorNames = new
ArrayList<String>();

for (Sensor sensor : mSensorList) {
    mSensorNames.add(sensor.getName());
}

adapter = new ArrayAdapter<String>(MainListActivity.
this,
    android.R.layout.simple_list_item_1,
mSensorNames);
setListAdapter(adapter);
```

Here, you get the instance of the Sensor service, and then get a list of available sensors using the *SensorManager* method *getSensorList()*. Set up an adapter for the list, and display it to the user. 

By: Saketkumar Srivastav

The author is an open source enthusiast, and an Android developer. Find him online at http://bit.ly/saket_blog and http://bit.ly/saket_wordpress.